

Appunti Avanzati in C++:

std::queue vs std::deque

1. std::queue: L'Adattatore di Contenitore (FIFO)

In C++, la `queue` non è una vera e propria struttura dati indipendente, ma un **Container Adaptor** (Adattatore di Contenitore). Questo significa che "avvolge" un altro contenitore (di default usa proprio una `deque`) e ne nasconde quasi tutti i metodi, esponendo solo quelli che garantiscono il rispetto rigoroso della politica **FIFO (First-In, First-Out)**.

Perché usare una queue?

Si usa quando l'algoritmo **esige** che gli elementi vengano elaborati nell'esatto ordine in cui sono arrivati (es. visita in ampiezza BFS di un grafo, code di stampa, simulazione sportelli). Nascondendo l'accesso tramite indice (`operator[]`), il C++ ti protegge dall'inserire o modificare accidentalmente elementi a metà della fila, garantendo la sicurezza logica del programma.

Metodi e Complessità ($O(1)$ per tutti)

- `push(const T& val)`: Aggiunge un elemento alla fine della coda.
- `pop()`: Rimuove il primo elemento. *Nota di design (Exception Safety)*: In C++, `pop()` restituisce `void`, non l'elemento rimosso. Se dovesse rimuovere e restituire il valore contemporaneamente, e avvenisse un'eccezione durante la copia del dato in uscita, il dato andrebbe perso.
- `front()`: Restituisce una *reference* al primo elemento (da leggere o modificare prima di fare `pop`).
- `back()`: Restituisce una *reference* all'ultimo elemento inserito.
- `empty()` e `size()`: Verificano se è vuota e ne restituiscono la dimensione.

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  int main() {
6      queue<string> clienti;
7
8      clienti.push("Mario"); // Coda: [Mario]
9      clienti.push("Luigi"); // Coda: [Mario, Luigi]
10     clienti.push("Peach"); // Coda: [Mario, Luigi, Peach]
11
12     // Il ciclo standard per elaborare e svuotare una queue
13     while (!clienti.empty()) {
14         // 1. Leggo chi e' il prossimo turno
15         string in_servizio = clienti.front();
16         cout << "Servendo: " << in_servizio << endl;
17
18         // 2. Lo rimuovo definitivamente dalla coda
19         clienti.pop();
20     }
21     // Stampa: Servendo Mario -> Servendo Luigi -> Servendo Peach
22     return 0;
23 }
```

2. std::deque: La Double-Ended Queue

La **deque** (pronunciata "deck") è una struttura dati potentissima e complessa. A differenza del **vector** (che usa un singolo blocco di memoria contiguo), la **deque** alloca la memoria in **blocchi separati (chunks)** di dimensione fissa, tenendo traccia di essi tramite un array centrale di puntatori.

Il vantaggio sulla gestione della memoria

Quando un **vector** si riempie e devi aggiungere un elemento, il programma deve cercare uno spazio in memoria più grande, copiare tutti i vecchi elementi e cancellare il vecchio array (*reallocation* molto costosa). Nella **deque**, se inserisci un elemento all'inizio (**push_front**) o alla fine (**push_back**) e lo spazio nel blocco corrente è finito, il C++ alloca semplicemente un **nuovo piccolo blocco** e lo collega agli altri, senza dover copiare o spostare nessun vecchio elemento!

Metodi e Complessità

- **push_back(val)** / **pop_back()**: Inserisce/Rimuove alla fine. *Complessità: $O(1)$.*
- **push_front(val)** / **pop_front()**: Inserisce/Rimuove all'inizio. *Complessità: $O(1)$* (nel **Vector** sarebbe $O(N)!$).
- **operator[i]** / **at(i)**: Accesso diretto all'i-esimo elemento. *Complessità: $O(1)$.*

```
1  #include <iostream>
2  #include <deque>
3  using namespace std;
4
5  int main() {
6      deque<int> d;
7
8      // Inserimenti super veloci O(1) in entrambe le direzioni
9      d.push_back(10); // [10]
10     d.push_back(20); // [10, 20]
11     d.push_front(5); // [5, 10, 20]
12
13     d.pop_back();    // Rimuove il 20. Resta: [5, 10]
14
15     // A differenza della queue, la deque espone l'accesso casuale e gli iteratori!
16     for(size_t i = 0; i < d.size(); i++) {
17         cout << d[i] << " "; // Stampa: 5 10
18     }
19
20     return 0;
21 }
```

3. Tabella Riassuntiva per l'Esame

Ecco quando scegliere quale contenitore:

Operazione	<code>std::vector</code>	<code>std::deque</code>	<code>std::queue</code>
Logica principale	Lista dinamica veloce.	Inserimenti in testa/- coda.	Fila rigorosa (FIFO).
Struttura in RAM	Singolo blocco contiguo.	Blocchi multipli frammentati.	Adattatore (nasconde i blocchi).
Accesso per indice [i]	Sì ($O(1)$ - Molto Veloce).	Sì ($O(1)$ - Leggermente più lento).	NO (Errore di compilazione).
Inserire in fondo (push_back)	Veloce (ma può causare riallocazione $O(N)$).	Sempre $O(1)$.	Sì (tramite <code>push()</code>).
Inserire in testa (push_front)	Lentissimo $O(N)$.	Sempre $O(1)$.	NO .